

Relatório de Auditoria de Segurança

DEPPI — Portal do Departamento de Extensão, Pesquisa, Pós-Graduação e Inovação

IFCE Campus Maracanaú

Repositório	c1c3ru/deppi_novo
Branch auditada	claude/security-audit-pdf-report-8eh5a6
Data	2026-08-29
Executado por	Claude Code (Sonnet 5) — auditoria assistida por IA
Categorias avaliadas	Isolamento · Permissões Frontend vs Backend · IDOR · Chaves Expostas · XSS

Isolamento	Permissões Frontend vs Backend	IDOR	Chaves Expostas	XSS
------------	--------------------------------	------	-----------------	-----

Sumário

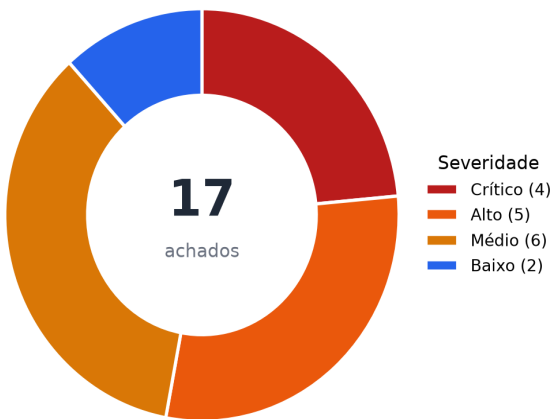
1. Resumo Executivo
2. Metodologia — Mapeamento de Stack e Categorias
3. Tabela de Achados Detalhados (arquivo:linha)
4. Achados por Categoria — Descrição Completa
5. Recomendações Priorizadas
6. Anexo — Templates de Issues para o GitHub

1. Resumo Executivo

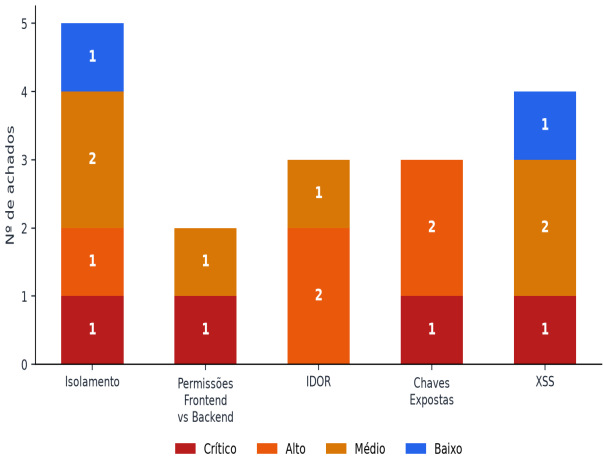
Esta auditoria cobriu as 5 categorias solicitadas (Isolamento, Permissões Frontend vs Backend, IDOR, Chaves Expostas e XSS) sobre o repositório **c1c3ru/deppi_novo** (branch **claudio/security-audit-pdf-report-8eh5a6**). Foram identificados **17 pontos fracos** (sendo **4 críticos** e **5 altos**) e **8 pontos fortes** confirmados, todos citados com arquivo e linha exatos.

O achado mais relevante é uma cadeia de exploração completa: qualquer uma das ~160 contas de servidores cadastradas recebe papel **admin** por padrão (ISO-1), nenhuma rota de escrita administrativa verifica papel no backend (PERM-1), e o conteúdo rico (Quill) de boletins/artigos é renderizado com `bypassSecurityTrustHtml` sem nenhuma sanitização (XSS-1). Como o token JWT fica em `localStorage`, um Stored XSS nessa cadeia resulta em sequestro de conta de qualquer visitante — não apenas um alerta de prova de conceito.

Distribuição de Achados por Severidade



Achados por Categoria (empilhado por severidade)



Mapeamento de Stack (visão geral)

Camada	Tecnologia
Frontend	Angular 20/21 (TypeScript, SCSS, NgRx, Standalone/Modules)
Backend	Node.js 18/20 + Express + TypeScript, Knex.js (query builder)
Banco de Dados	PostgreSQL 15 (via Docker) + migrações/seeds Knex
Cache/Fila	Redis 7 (Docker)
Orquestração	Docker Compose (app, postgres, redis, prometheus, grafana, backup) + Nginx + PM2
Legado	PHP 7/8 + Composer (mpdf/mpdf) — módulo cargahoraria/ (PIT/RIT)
Scripts	Python 3 (bin/sync_talentos.py)
Autenticação	JWT (jsonwebtoken) + bcryptjs, sem OAuth/Supabase/Firebase

2. Metodologia — Mapeamento de Stack e Categorias

Antes de procurar falhas, a stack real do projeto foi mapeada (ver tabela completa abaixo) e cada uma das 5 categorias exigidas foi redefinida no contexto específico dessa stack — este projeto não usa Supabase/Firebase nem Flutter; é um monólito Angular + Node/Express + Knex/PostgreSQL, com um módulo legado em PHP.

Stack completa

Camada	Tecnologia	Relevância para a auditoria
Frontend	Angular 20/21 (TypeScript, SCSS, NgRx, Standalone/Modules)	Onde vivem os guards de rota (RoleGuard/AuthGuard), o binding de HTML do editor Quill e o armazenamento do JWT (localStorage).
Backend	Node.js 18/20 + Express + TypeScript, Knex.js (query builder)	Onde a autorização REAL deveria ser reforçada (middlewares authMiddleware/authorizeRoles) e onde o acesso a objetos por ID é controlado (ou não).
Banco de Dados	PostgreSQL 15 (via Docker) + migrações/seeds Knex	Onde vivem os dados sensíveis (senhas, PII de visitas escolares) e os limites (ou ausência deles) de isolamento entre registros.
Cache/Fila	Redis 7 (Docker)	Provisionado para sessões/blacklist de token, mas não utilizado pelo código — relevante para Isolamento de sessão.
Orquestração	Docker Compose (app, postgres, redis, prometheus, grafana, backup) + Nginx + PM2	Onde vivem isolamento de rede/containers, exposição de portas e chaves de fallback hardcoded.
Legado	PHP 7/8 + Composer (mpdf/mpdf) — módulo cargahoraria/ (PIT/RIT)	Aplicação PHP standalone, sem framework, que gera PDFs a partir de formulários — superfície separada para XSS/HTML injection.
Scripts	Python 3 (bin/sync_talentos.py)	Consome planilha pública do Google Sheets (publicada por design) para gerar src/assets/data/talentos.json; sem chaves de API envolvidas.
Autenticação	JWT (jsonwebtoken) + bcryptjs, sem OAuth/Supabase/Firebase	Define o modelo de Permissões (roles no payload do JWT) e o raio de impacto de XSS (token acessível via JS).

Como cada categoria foi interpretada nesta stack

Isolamento

Nesta stack (sem multi-tenancy SaaS/RLS do Supabase), 'Isolamento' foi interpretado como: (1) separação de privilégios entre papéis de usuário (RBAC real, não só 'logado ou não'); (2) separação de segredos por finalidade (access token vs refresh token); (3) isolamento entre recursos de negócio (laboratório A não deveria ser controlável por qualquer usuário como se fosse o dono); (4) isolamento de rede/processo em Docker (containers, portas expostas, usuário não-root).

Permissões Frontend vs Backend

Angular usa AuthGuard/RoleGuard client-side para esconder rotas e botões administrativos. A pergunta auditada foi: para cada rota de escrita (POST/PUT/PATCH/DELETE) do Express, existe um middleware equivalente (authMiddleware + authorizeRoles) que reforce no servidor a mesma regra que o frontend aplica na UI? Resultado: não — authorizeRoles existe mas nunca é importado/usado.

IDOR

Para cada endpoint que recebe um :id (ou :lab_id) via URL/params, foi verificado se o backend confirma que o usuário autenticado tem relação legítima com aquele registro específico (dono, participante, coordenador) antes

de ler/alterar/apagar — em vez de apenas checar 'existe um JWT válido'.

Chaves Expostas

Sem Supabase/Firebase (sem anon key/service key no bundle do frontend), o foco foi: segredos hardcoded em código versionado (seeds, docker-compose, Dockerfile), fallback inseguro de variáveis de ambiente, e presença/ausência de arquivos .env no histórico do Git.

XSS

Angular sanitiza [innerHTML] por padrão — o risco real está em usos explícitos de DomSanitizer.bypassSecurityTrustHtml (que desativa a proteção) sobre conteúdo gerado pelo editor rico Quill e persistido no backend sem sanitização (nem client-side com DOMPurify, nem server-side). Também avaliado no módulo legado PHP (interpolação direta de \$_POST em HTML/PDF).

Critérios de severidade

Severidade	Critério
Crítico	Compromete confidencialidade/integridade de todo o sistema ou de muitas contas, com baixo esforço e sem pré-condições especiais.
Alto	Impacto sério, mas depende de uma pré-condição razoável (ex.: deploy específico, uma etapa adicional) ou atinge um subconjunto relevante de dados/usuários.
Médio	Falha real, porém com raio de impacto limitado ou que exige uma condição menos provável para ser explorada.
Baixo	Fragilidade de reforço (defense-in-depth); não é explorável de forma independente hoje, mas deveria ser corrigida.
Informativo	Ponto forte confirmado ou observação sem risco de segurança direto.

3. Tabela de Achados Detalhados (arquivo:linha)

Cada linha referencia o(s) arquivo(s) e a(s) linha(s) exata(s) onde o achado foi observado. Achados do tipo **Fraqueza** e **Ponto Forte** aparecem juntos, ordenados por categoria e severidade.

ID	Categoria	Sev.	Arquivo:Linha	Achado
KEY-1	Chaves Expostas	Crítico	backend/src/database/seeds/001_initial_data.ts:13, 811-822	Senha de admin mestre hardcoded no seed padrão (matrícula 123456)
KEY-2	Chaves Expostas	Alto	docker-compose.yml:26, 68	Senha do PostgreSQL com fallback hardcoded no docker-compose.yml versionado
KEY-3	Chaves Expostas	Alto	Dockerfile:58 (+2 arquivo(s))	Dockerfile grava .env.example (com segredos-placeholder públicos) como .env real dentro da imagem publicada
KEY-S1	Chaves Expostas	Informativo	.gitignore:14-17 (+1 arquivo(s))	Arquivo .env real nunca foi commitado (apenas .env.example, corretamente listado no .gitignore)
KEY-S2	Chaves Expostas	Informativo	backend/src/routes/auth.routes.ts:93-96, 308 (+1 arquivo(s))	Senhas de usuário armazenadas com bcrypt (custo 12); frontend sem segredos reais
IDOR-1	IDOR	Alto	backend/src/routes/upload.routes.ts:170-179	DELETE /api/upload/:id apaga qualquer anexo de qualquer usuário — ID sequencial, sem checar proprietário
IDOR-2	IDOR	Alto	backend/src/controllers/visit.controller.ts:12-22, 77-103	GET /api/visitas/:id expõe PII completa de escolas sem checar relação do requisitante com a visita
IDOR-3	IDOR	Médio	backend/src/controllers/visit.controller.ts:195-270, 272-292	PATCH /api/visitas/:id/status e /:id/labs/:lab_id alteram qualquer visita/lab sem checar posse
IDOR-S1	IDOR	Informativo	backend/src/routes/upload.routes.ts:42-51	Nomes de arquivo de upload são aleatórios (não previsíveis)
ISO-1	Isolamento	Crítico	backend/src/database/seeds/001_initial_data.ts:12-13, 799-822	Nenhum isolamento real de privilégios: todo usuário autenticado é tratado como admin
ISO-2	Isolamento	Alto	backend/src/routes/auth.routes.ts:115-123, 179-182	Access token e refresh token compartilham o mesmo segredo (JWT_REFRESH_SECRET nunca é usado)
ISO-3	Isolamento	Médio	backend/src/database/migrations/006_create_laboratorios_table.ts:4-13 (+1 arquivo(s))	Laboratórios não têm proprietário/coordenador — qualquer usuário controla a disponibilidade de qualquer lab
ISO-4	Isolamento	Médio	docker-compose.yml:119-120, 139-143	Prometheus exposto sem autenticação e Grafana com senha padrão publicados diretamente no host
ISO-5	Isolamento	Baixo	backend/src/routes/auth.routes.ts:244-255	Logout não revoga sessão — Redis provisionado mas nunca utilizado pelo backend

ID	Categoria	Sev.	Arquivo:Linha	Achado
ISO-S1	Isolamento	Informativo	Dockerfile:42-43, 73	Container roda como usuário não-root
ISO-S2	Isolamento	Informativo	docker-compose.yml:60-113, 192-197	Rede Docker dedicada e Postgres/Redis não publicam portas no host
PERM-1	Permissões Frontend vs Backend	Crítico	backend/src/routes/laboratorios.routes.ts:9-11 (+2 arquivo(s))	Rotas de escrita administrativas exigem apenas login (authMiddleware), nunca papel (authorizeRoles)
PERM-2	Permissões Frontend vs Backend	Médio	backend/src/routes/boletim.routes.ts:16-18 (+1 arquivo(s))	Listagem administrativa de boletins/edições (rascunhos inclusos) exige só autenticação, não papel
PERM-S1	Permissões Frontend vs Backend	Informativo	backend/src/routes/user.routes.ts:9-31, 36-58	Endpoints de perfil sempre operam sobre req.user.id do JWT, nunca sobre ID vindo do cliente
XSS-1	XSS	Crítico	src/app/features/boletins/components/boletim-detail/boletim-detail.component.ts:576-578 (+2 arquivo(s))	Stored XSS via conteúdo do editor Quill em Boletins e Revista (bypassSecurityTrustHtml sem nenhuma sanitização)
XSS-2	XSS	Médio	backend/src/index.ts:60-77 (+1 arquivo(s))	CSP do Helmet nunca alcança a página real (SPA é servida direto pelo Nginx, sem os mesmos headers)
XSS-3	XSS	Médio	cargahoraria/PIT/relatorioPIT.php:7-14, 33-188 (exemplo), 548-551	Módulo legado PHP (cargahoraria) interpola \$_POST direto em HTML enviado ao mPDF, sem escaping
XSS-4	XSS	Baixo	src/app/features/pit-rit/pit-rit.module.ts:23 (+1 arquivo(s))	innerHTML vinculado a strings de tradução estáticas (não explorável hoje, mas padrão frágil)
XSS-S1	XSS	Informativo	src/app/features/revista/components/artigo-detail/artigo-detail.component.ts:39-41	Interpolação padrão do Angular ({{{}}}) usada corretamente para campos de texto simples
XSS-S2	XSS	Informativo	backend/src/routes/upload.routes.ts:23-36, 60-68	Whitelist estrita de MIME+extensão no upload bloqueia .html/.svg/.js

4. Achados por Categoria — Descrição Completa

Isolamento

[ISO-1] Nenhum isolamento real de privilégios: todo usuário autenticado é tratado como admin

CRÍTICO

ACHADO · Arquivo(s): backend/src/database/seeds/001_initial_data.ts — Linha(s): 12-13, 799-822

```
const adminPasswordHash = await bcrypt.hash(/* segredo real – redigido, ver linha citada */, 12);
... roles: JSON.stringify(['user', 'admin']), // All allowed to edit bulletins
```

Descrição: O seed padrão (executado no setup documentado no README) atribui roles ['user','admin'] a TODOS os ~160 usuários da lista `allowedUsers` e também ao admin mestre. Não existe nenhum usuário 'somente leitura' ou 'somente user'. Além disso, o middleware `authorizeRoles()` (backend/src/middleware/auth.middleware.ts:62) que faria a checagem de papel nunca é importado ou usado em nenhuma rota (confirmado via busca em todo backend/src). Ou seja: o conceito de 'papel' existe no schema (coluna jsonb roles) mas não existe em nenhum ponto de aplicação real.

Impacto: Qualquer uma das ~160 contas de servidores do IFCE (ou a conta admin mestre) tem, na prática, controle total sobre boletins, laboratórios, revista e aprovação de visitas escolares. Não há segmentação de menor privilégio.

Recomendação: Aplicar `authorizeRoles('admin')` (ou papel equivalente) em todas as rotas de escrita administrativa; redesenhar o seed para atribuir 'admin' apenas a contas explícitas; adicionar teste de regressão que falhe se uma rota de escrita não tiver `authorizeRoles`.

[ISO-2] Access token e refresh token compartilham o mesmo segredo (JWT_REFRESH_SECRET nunca é usado)

ALTO

ACHADO · Arquivo(s): backend/src/routes/auth.routes.ts — Linha(s): 115-123, 179-182

```
const accessToken = jwt.sign(..., config.jwt.secret, { expiresIn: '1h' });
const refreshToken = jwt.sign({ id: user.id }, config.jwt.secret, { expiresIn: '30d' });
// /auth/refresh:
const decoded = jwt.verify(refreshToken, config.jwt.secret) as jwt.JwtPayload;
```

Descrição: `backend/src/config/environment.ts:42-47` define `config.jwt.refreshSecret` separadamente, mas em nenhum lugar do código esse valor é lido — tanto o access token quanto o refresh token de 30 dias são assinados e verificados com `config.jwt.secret`. Nenhum claim (`type: 'access' | 'refresh'`) distingue os dois. Isso significa que um access token de curta duração pode ser reenviado como refresh token em POST /auth/refresh, e que comprometer o segredo único compromete ambos os tipos de token permanentemente, sem possibilidade de rotacionar um sem o outro.

Impacto: Perde-se o isolamento de blast-radius entre tokens de curta e longa duração; incidentes de resposta (revogar só refresh tokens) tornam-se impossíveis.

Recomendação: Assinar/verificar refresh tokens com `config.jwt.refreshSecret`; incluir um claim `type` e validá-lo em cada endpoint; considerar rotacionar o refresh token a cada uso.

[ISO-3] Laboratórios não têm proprietário/coordenador — qualquer usuário controla a disponibilidade de qualquer lab

MÉDIO

ACHADO · Arquivo(s): backend/src/database/migrations/006_create_laboratorios_table.ts; backend/src/controllers/visit.controller.ts — Linha(s): 006_create_laboratorios_table.ts:4-13; visit.controller.ts:272-292

```
// migration: laboratorios não tem coluna owner/coordinator_id
// controller:
async updateLabAvailability(req, res, next) {
  const { id, lab_id } = req.params;
  const { status } = req.body;
  const updated_by_user_id = (req as any).user?.id || null;
  const [updated] = await db('visit_lab_availability')
    .where({ visit_id: id, lab_id })
    .update({ status, updated_by_user_id, ... });
}
```

Descrição: A tabela `laboratorios` não possui nenhuma coluna que associe um laboratório a um coordenador/responsável. O endpoint `PATCH /api/visitas/:id/labs/:lab\_id` (protegido só por `authMiddleware`) permite que QUALQUER usuário autenticado marque QUALQUER laboratório como disponível/indisponível para QUALQUER visita, sem vinculação ao seu próprio laboratório.

Impacto: Um funcionário de um laboratório pode confirmar ou recusar disponibilidade em nome de outro laboratório, inclusive influenciando o e-mail de confirmação automático enviado à escola.

Recomendação: Adicionar uma tabela/coluna de vínculo usuário↔laboratório e validar, no controller, que `req.user.id` coordena o `lab\_id` informado (ou tem papel admin).

[ISO-4] Prometheus exposto sem autenticação e Grafana com senha padrão publicados diretamente no host

MÉDIO

ACHADO · Arquivo(s): docker-compose.yml — Linha(s): 119-120, 139-143

```
prometheus:
  ports: ["9090:9090"]
...
grafana:
  ports: ["3001:3000"]
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD:-admin}
```

Descrição: Prometheus (sem autenticação nativa) e Grafana são publicados diretamente em portas do host, fora do Nginx/reverse proxy. Se `GRAFANA\_PASSWORD` não for definido no ambiente, o Grafana sobe com a senha padrão 'admin', divulgada no próprio docker-compose.yml versionado.

Impacto: Exposição de métricas internas (Prometheus) e possível acesso administrativo ao Grafana (dashboards, datasources) para qualquer host com rota de rede até as portas 9090/3001.

Recomendação: Não publicar essas portas diretamente (usar rede interna do Compose + proxy autenticado, ou túnel/VPN); remover o fallback `:-admin` e exigir `GRAFANA\_PASSWORD` obrigatório.

[ISO-5] Logout não revoga sessão — Redis provisionado mas nunca utilizado pelo backend

BAIXO

ACHADO · Arquivo(s): backend/src/routes/auth.routes.ts — Linha(s): 244-255

```
router.post('/logout', [...], async (req, res) => {  
  // TODO: Se usar Redis ou Blacklist no banco, revogar o token aqui  
  res.json({ message: 'Logout realizado com sucesso' });  
});
```

Descrição: Redis está provisionado em `docker-compose.yml` e configurado em `backend/src/config/environment.ts`, mas uma busca em todo `backend/src` mostra que nunca é efetivamente utilizado (nenhum client Redis é instanciado). O logout apenas responde 200 OK sem invalidar nada no servidor; o token de acesso continua válido até expirar (1h) e o refresh token continua válido por 30 dias mesmo após 'logout'.

Impacto: Um token vazado (ex.: via XSS, veja XSS-1) continua utilizável mesmo depois que a vítima faz logout ou troca de senha.

Recomendação: Implementar blacklist/allowlist de refresh tokens no Redis já disponível, invalidada no logout e na troca de senha.

[ISO-S1] Container roda como usuário não-root

INFORMATIVO

PONTO FORTE · Arquivo(s): Dockerfile — Linha(s): 42-43, 73

```
RUN addgroup --system --gid 1001 nodejs  
RUN adduser --system --uid 1001 nextjs  
...  
USER nextjs
```

Descrição: A imagem de produção cria um usuário de sistema dedicado e troca para ele antes do ENTRYPOINT, reduzindo o impacto de uma eventual RCE dentro do container.

Impacto: Reforça isolamento de privilégios no nível de processo/container.

Recomendação: Manter a prática; considerar também `--read-only` filesystem e `cap-drop: ALL` no compose.

[ISO-S2] Rede Docker dedicada e Postgres/Redis não publicam portas no host

INFORMATIVO

PONTO FORTE · Arquivo(s): docker-compose.yml — Linha(s): 60-113, 192-197

```
networks:  
  deppi-network:  
    driver: bridge  
    ipam:  
      config:  
        - subnet: 172.20.0.0/16  
# postgres/redis: sem bloco 'ports'
```

Descrição: Postgres e Redis só são acessíveis dentro da rede `deppi-network`; apenas `app`, `prometheus` e `grafana` publicam portas no host (as duas últimas com ressalvas, ver ISO-4).

Impacto: Reduz a superfície de ataque direta ao banco de dados e ao cache.

Recomendação: Manter; aplicar o mesmo padrão às portas de monitoramento (ISO-4).

Permissões Frontend vs Backend

[PERM-1] Rotas de escrita administrativas exigem apenas login (authMiddleware), nunca papel (authorizeRoles)

CRÍTICO

ACHADO · Arquivo(s): backend/src/routes/laboratorios.routes.ts; backend/src/routes/boletim.routes.ts; backend/src/routes/revista.routes.ts — Linha(s): laboratorios.routes.ts:9-11; boletim.routes.ts:215,259,297; revista.routes.ts:119,163,209,261,307,334

```
router.post('/', authMiddleware, laboratorioController.create);
router.put('/:id', authMiddleware, laboratorioController.update);
router.delete('/:id', authMiddleware, laboratorioController.delete);
// idem em boletim.routes.ts e revista.routes.ts – nunca 'authorizeRoles(...)'
```

Descrição: O frontend Angular usa `RoleGuard`/AuthService.hasAnyRole()` (src/app/core/guards/role.guard.ts) para esconder as telas de administração de quem não tem o papel esperado. Porém, nenhuma das rotas Express equivalentes chama `authorizeRoles(...)`; todas usam apenas `authMiddleware`, que só verifica 'existe um JWT válido'. Isso quebra a premissa de que o frontend e o backend aplicam a mesma política.

Impacto: Qualquer conta autenticada pode chamar diretamente (curl/Postman) `POST/PUT/DELETE /api/laboratorios` , ` /api/boletins` e ` /api/revista/...``, contornando totalmente a UI administrativa, mesmo que o RoleGuard nunca mostre esses botões para ela.

Recomendação: Adicionar `authorizeRoles('admin')` (ou papel adequado) em cada rota de escrita, espelhando exatamente a regra usada em `role.guard.ts``; cobrir com teste de integração que autentica como usuário comum e espera 403.

[PERM-2] Listagem administrativa de boletins/edições (rascunhos inclusos) exige só autenticação, não papel

MÉDIO

ACHADO · Arquivo(s): backend/src/routes/boletim.routes.ts; backend/src/routes/revista.routes.ts — Linha(s): boletim.routes.ts:16-18; revista.routes.ts:45-48

```
router.get('/admin/all', authMiddleware, async (req, res) => { ... });
router.get('/edicoes/admin/all', authMiddleware, async (req, res) => { ... });
```

Descrição: Os endpoints `/admin/all`` retornam boletins e edições em qualquer status (incluindo 'draft'), protegidos só por `authMiddleware``. Qualquer conta autenticada vê conteúdo ainda não publicado.

Impacto: Vazamento de informação (rascunhos internos) para contas que a UI nunca deveria expor a essas telas.

Recomendação: Aplicar `authorizeRoles('admin')` também nessas rotas de leitura administrativa.

[PERM-S1] Endpoints de perfil sempre operam sobre req.user.id do JWT, nunca sobre ID vindo do cliente

INFORMATIVO

PONTO FORTE · Arquivo(s): backend/src/routes/user.routes.ts — Linha(s): 9-31, 36-58

```
router.get('/profile', async (req, res) => {
  const user = await db('users').where({ id: req.user!.id }).first();
  ...
  router.put('/profile', async (req, res) => {
    await db('users').where({ id: req.user!.id }).update({ ... });
  });
});
```

Descrição: Diferente das rotas administrativas, o módulo de perfil nunca aceita um ``:id`` de URL/body para decidir qual usuário ler/atualizar — sempre usa o ``id`` decodificado do JWT assinado pelo servidor.

Impacto: Padrão correto: elimina qualquer possibilidade de um usuário alterar o perfil de outro via manipulação de parâmetro.

Recomendação: Usar este módulo como referência ao corrigir os demais controllers.

IDOR

[IDOR-1] DELETE /api/upload/:id apaga qualquer anexo de qualquer usuário — ID sequencial, sem checar proprietário

ALTO

ACHADO · Arquivo(s): backend/src/routes/upload.routes.ts — Linha(s): 170-179

```
router.delete('/:id', async (req: Request, res: Response) => {
  const { id } = req.params;
  await db('uploads').where({ id }).update({ is_active: false });
  res.json({ success: true, message: 'Anexo removido' });
});
```

Descrição: A tabela `uploads` usa `table.increments('id')`

(backend/src/database/migrations/005_create_uploads_table.ts:5) — IDs inteiros sequenciais e previsíveis (1, 2, 3, ...). A coluna `uploaded\_by` existe no schema, mas o handler de DELETE nunca a compara com `req.user.id`; só exige que exista um JWT válido (o router inteiro fica atrás de `authMiddleware` aplicado em backend/src/index.ts:127).

Impacto: Qualquer conta autenticada pode iterar `DELETE /api/upload/1`, `/2`, `/3`... e desativar anexos de boletins/laboratórios que não enviou, quebrando imagens/documentos publicados por outras pessoas.

Recomendação: Comparar `upload.uploaded\_by === req.user.id` (ou exigir papel admin) antes de apagar; validar `id` numérico antes da query.

[IDOR-2] GET /api/visitas/:id expõe PII completa de escolas sem checar relação do requisitante com a visita

ALTO

ACHADO · Arquivo(s): backend/src/controllers/visit.controller.ts — Linha(s): 12-22, 77-103

```
function toPublicVisit(visit) {
  return { id: visit.id, school_name: ..., ... }; // inclui 'id'
}
...
async getById(req, res, next) {
  const { id } = req.params;
  const visit = await db('school_visits').where({ id }).first();
  ...
  res.json({ ...visit, labs });
}
```

Descrição: `school\_visits.id` é UUID (mitiga adivinhação à força bruta — ponto positivo), mas o próprio endpoint público não autenticado `GET /api/visitas` já devolve esse UUID em `toPublicVisit()` para visitas confirmadas. A partir daí, `GET /api/visitas/:id` (só `authMiddleware`, sem checar papel ou vínculo com a visita) devolve o registro completo: `contact\_email`, `contact\_phone`, `director\_name`, `whatsapp\_phone`, `institutional\_email`, `notes`, etc.

Impacto: Qualquer uma das ~160 contas autenticadas pode pegar um UUID visto anonimamente na agenda pública e obter dados de contato pessoal de diretores/responsáveis escolares, sem qualquer relação legítima com aquele agendamento específico.

Recomendação: Restringir campos sensíveis em `getById` a usuários com papel administrativo/gestão de visitas; ou exigir que o UUID completo seja tratado como capability token de uso único, não reexposto na listagem pública.

[IDOR-3] PATCH /api/visitas/:id/status e /:id/labs/:lab_id alteram qualquer visita/lab sem checar posse

MÉDIO

ACHADO · Arquivo(s): backend/src/controllers/visit.controller.ts — Linha(s): 195-270, 272-292

```
async updateStatus(req, res, next) {  
  const { id } = req.params;  
  const { status } = req.body;  
  const visit = await db('school_visits').where({ id }).first();  
  ... // nenhuma checagem de que req.user está autorizado para ESTA visita
```

Descrição: Assim como ISO-3, estes endpoints só exigem `authMiddleware`. Qualquer conta pode confirmar/cancelar qualquer visita escolar (o que dispara e-mail automático ao contato da escola e cria evento no Google Calendar institucional).

Impacto: Ações de negócio sensíveis (aprovar visita, notificar escola, ocupar agenda) não estão restritas a quem deveria geri-las.

Recomendação: Mesma correção de PERM-1/ISO-3: introduzir papel/vínculo específico para gestão de visitas.

[IDOR-S1] Nomes de arquivo de upload são aleatórios (não previsíveis)

INFORMATIVO

PONTO FORTE · Arquivo(s): backend/src/routes/upload.routes.ts — Linha(s): 42-51

```
filename: (_req, file, cb) => {  
  const uniqueName = `${Date.now()}-${Math.round(Math.random() * 1e9)}${safeExt}`;  
  cb(null, uniqueName);  
}
```

Descrição: Mesmo com o registro no banco sendo vulnerável a IDOR (IDOR-1), o caminho físico do arquivo em `/uploads/` não é o nome original nem sequencial, dificultando adivinhação direta via URL estática.

Impacto: Camada adicional de proteção contra enumeração de arquivos pelo caminho estático do Nginx/Express.

Recomendação: Manter; garantir que a mesma aleatoriedade seja usada caso o storage migre para um bucket externo.

Chaves Expostas

[KEY-1] Senha de admin mestre hardcoded no seed padrão (matrícula 123456)

CRÍTICO

ACHADO · Arquivo(s): backend/src/database/seeds/001_initial_data.ts — Linha(s): 13, 811-822

```
const adminPasswordHash = await bcrypt.hash(/* segredo real – redigido, ver linha citada */, 12);
...
usersToInsert.push({
  registration: '123456',
  name: 'Administrador Sistema',
  password_hash: adminPasswordHash,
  roles: JSON.stringify(['user', 'admin']),
  ...
});
```

Descrição: O seed é executado como etapa padrão de instalação (README.md, seção 'Build do Backend', comando `npm run seed`), sem nenhuma checagem de `NODE\_ENV`. A senha em texto plano (valor real redigido neste relatório — ver linha citada no repositório) fica permanentemente no histórico do repositório público `c1c3ru/deppi\_novo`, associada a uma matrícula fixa e conhecida ('123456').

Impacto: Qualquer pessoa com acesso de leitura ao repositório (público) conhece uma credencial válida de administrador caso o seed seja aplicado em produção sem troca de senha posterior.

Recomendação: Gerar senha aleatória no primeiro seed e imprimir-la uma única vez no log (ou exigir troca no primeiro login); nunca hardcodar credenciais reais em seeds versionados; adicionar guarda `if (NODE\_ENV==='production') throw` para seeds de dados de teste.

[KEY-2] Senha do PostgreSQL com fallback hardcoded no docker-compose.yml versionado

ALTO

ACHADO · Arquivo(s): docker-compose.yml — Linha(s): 26, 68

```
- POSTGRES_PASSWORD=${DB_PASSWORD:-«segredo redigido»} # Fallback caso a variável falhe
```

Descrição: Caso a variável de ambiente `DB\_PASSWORD` não esteja definida no momento de `docker-compose up`, o Postgres é iniciado com uma senha literal fixa (valor real redigido neste relatório — ver linha citada no repositório), publicada no repositório.

Impacto: Uma instalação apressada (sem `.env` completo) sobe um banco com senha conhecida publicamente.

Recomendação: Remover o fallback; fazer o compose falhar explicitamente se `DB\_PASSWORD` não estiver definido (`\${DB\_PASSWORD:?obrigatório}`).

[KEY-3] Dockerfile grava .env.example (com segredos-placeholder públicos) como .env real dentro da imagem publicada

ALTO

ACHADO · Arquivo(s): Dockerfile; backend/src/config/environment.ts; .env.example — Linha(s): Dockerfile:58; environment.ts:9-15; .env.example:21,23

```
# Dockerfile
COPY --from=builder --chown=nextjs:nodejs /app/.env.example /app/.env

# .env.example
JWT_SECRET=your_super_secret_jwt_key_here_change_in_production

# environment.ts
function requireEnv(key, fallback) {
  const value = process.env[key] || fallback;
  if (!value && process.env.NODE_ENV === 'production') throw new Error(...);
  return value ?? '';
}
```

Descrição: A imagem `ghcr.io/deppimaracauan/deppi\_novo:main` (referenciada diretamente em docker-compose.yml:6) contém, dentro dela, um arquivo `app/.env` que é cópia literal do `.env.example` público. `dotenv.config()` só preenche uma chave que AINDA NÃO existe em `process.env`. Se o container for executado sem TODAS as variáveis passadas explicitamente (ex.: `docker run` direto da imagem publicada, sem o docker-compose.yml completo), `process.env.JWT\_SECRET` é preenchido pelo dotenv com o placeholder público `your\_super\_secret\_jwt\_key\_here\_change\_in\_production`. Como esse valor é uma string não-vazia, o guard `requireEnv` (pensado para falhar em produção sem segredo) é satisfeito silenciosamente por um segredo já conhecido publicamente.

Impacto: Em um deploy que não injete `JWT\_SECRET` de forma explícita, a aplicação assina JWTs de produção com um segredo publicado no GitHub — permitindo forjar tokens de qualquer usuário/admin.

Recomendação: Não copiar `.env.example` para `.env` na imagem final; se um arquivo de exemplo for necessário, nomear diferente de `.env` para que o dotenv nunca o carregue automaticamente; validar no boot que segredos críticos não correspondem aos valores literais do `.env.example`.

[KEY-S1] Arquivo .env real nunca foi commitado (apenas .env.example, corretamente listado no .gitignore)

INFORMATIVO

PONTO FORTE · Arquivo(s): .gitignore; git history — Linha(s): .gitignore:14-17

```
$ git ls-files | grep -iE '\.env' → apenas .env.example
$ git log --all --diff-filter=A --name-only | grep -E '\.env$' → (vazio)
```

Descrição: Verificação direta do histórico do Git confirma que nenhum `.env` real (com segredos de produção de fato) jamais foi adicionado ao repositório; `.gitignore` cobre `.env`, `.env.local` e `\*.env.\*`.

Impacto: Reduz drasticamente o risco de vazamento de segredos reais via histórico do Git (o risco remanescente está nos fallbacks hardcoded documentados em KEY-1/2/3).

Recomendação: Manter a disciplina; considerar `git-secrets`/`gitleaks` no CI para reforçar continuamente.

[KEY-S2] Senhas de usuário armazenadas com bcrypt (custo 12); frontend sem segredos reais

INFORMATIVO

PONTO FORTE · Arquivo(s): backend/src/routes/auth.routes.ts; src/environments/environment.prod.ts
— Linha(s): auth.routes.ts:93-96,308; environment.prod.ts:1-5

```
const isValidPassword = await bcrypt.compare(password, user.password_hash);  
...  
const hashedPassword = await bcrypt.hash(rawPassword, 12);  
// environment.prod.ts só contém apiUrl, googleAnalyticsId (público) e sentryDsn (placeholder)
```

Descrição: Hashing de senha usa bcrypt com fator de custo 12 (adequado). Os arquivos de ambiente do Angular não contêm nenhuma chave privada — apenas valores que são inerentemente públicos no bundle de um SPA (Analytics ID, DSN do Sentry).

Impacto: Boas práticas de hashing; superfície de 'chaves no bundle do frontend' bem controlada.

Recomendação: Manter; ao configurar Sentry real, usar DSN real (que é seguro expor publicamente por design).

XSS

[XSS-1] Stored XSS via conteúdo do editor Quill em Boletins e Revista (bypassSecurityTrustHtml sem nenhuma sanitização)

CRÍTICO

ACHADO · Arquivo(s):

```
src/app/features/boletins/components/boletim-detail/boletim-detail.component.ts;  
src/app/features/revista/components/artigo-detail/artigo-detail.component.ts;  
backend/src/routes/boletim.routes.ts — Linha(s): boletim-detail.component.ts:576-578;  
artigo-detail.component.ts:176-178; boletim.routes.ts:282  
  
// boletim-detail.component.ts  
// Sanitize HTML content from Quill to prevent XSS  
this.safeContent = b.content ? this.sanitizer.bypassSecurityTrustHtml(b.content) : null;  
  
// backend/src/routes/boletim.routes.ts (PUT)  
if (content !== undefined) updateData.content = content; // sem sanitização
```

Descrição: `bypassSecurityTrustHtml` faz exatamente o oposto do que o comentário do código diz: desativa o sanitizador automático do Angular para aquele valor. O backend também nunca sanitiza `content` ao salvar (nem ao criar, nem ao atualizar). Uma busca por `DOMPurify`/`dompurify` em `src/app` não encontra nenhum uso — a dependência só existe como `overrides` transitivo em package.json, nunca chamada pela aplicação. O mesmo padrão se repete em `artigo-detail.component.ts` para artigos da Revista.

Impacto: Combinado com PERM-1 (qualquer uma das ~160 contas autenticadas pode publicar boletins/artigos) e com o armazenamento do JWT em localStorage (src/app/core/services/storage.service.ts), um payload como `

Recomendação: Sanitizar `content` no backend antes de persistir (ex.: `sanitize-html`/sanitizador Quill-aware com allowlist de tags); no frontend, trocar `bypassSecurityTrustHtml` por `DomSanitizer.sanitize(SecurityContext.HTML, content)` ou aplicar DOMPurify explicitamente antes do bind; mover o JWT para cookie httpOnly+Secure+SameSite.

[XSS-2] CSP do Helmet nunca alcança a página real (SPA é servida direto pelo Nginx, sem os mesmos headers)

MÉDIO

ACHADO · Arquivo(s): backend/src/index.ts; nginx.conf — Linha(s): index.ts:60-77; nginx.conf:30-39

```
// index.ts – só decora respostas do Express (/api/*, /health)
this.app.use(helmet({
  contentSecurityPolicy: { directives: { scriptSrc: ['"self"'], ... } }
}));

// nginx.conf – serve o SPA direto do disco, sem headers equivalentes
location / {
  try_files $uri $uri/ /index.html; // root /app/dist/deppi
}
```

Descrição: Em produção, o Nginx serve `index.html` e os bundles JS/CSS diretamente do disco (`root /app/dist/deppi`), sem passar pelo Express/Helmet. O CSP restritivo configurado (`scriptSrc: ["self"]`) só é enviado em respostas JSON de `/api/\*`, que o navegador nunca renderiza como HTML — ou seja, a página onde um XSS (XSS-1) realmente executaria não recebe CSP nenhum.

Impacto: Uma camada de defesa em profundidade que mitigaria/bloquearia a execução do payload de XSS-1 está configurada, mas nunca chega a ser aplicada onde importa.

Recomendação: Adicionar os mesmos headers de segurança (CSP, X-Content-Type-Options, etc.) diretamente no bloco `server` do Nginx que serve `/`, ou roteá-lo através do Express.

[XSS-3] Módulo legado PHP (cargahoraria) interpola \$_POST direto em HTML enviado ao mPDF, sem escaping

MÉDIO

ACHADO · Arquivo(s): cargahoraria/PIT/relatorioPIT.php — Linha(s): 7-14, 33-188 (exemplo), 548-551

```
setcookie("nome", $_POST['nome'], time() + (86400 * 30), "/");
...
$html = '... <th class="celula2">' . $_POST['nome'] . '</tr> ...';
...
$pdf->WriteHTML($html);
$pdf->Output('relatorioPIT.pdf', 'I');
```

Descrição: Nenhuma ocorrência de `htmlspecialchars`/`htmlentities` existe no arquivo (confirmado por busca). Todos os campos do formulário PIT (nome, SIAPE, e-mail, curso, campus, etc.) vão direto de `\$\_POST` para uma string HTML montada por concatenação e passada a `mPDF::WriteHTML()`. Os mesmos valores também são gravados via `setcookie()` sem as flags `HttpOnly`/`Secure`/`SameSite`.

Impacto: Um valor malicioso injeta HTML arbitrário no PDF gerado (falsificação de conteúdo de um documento institucional) e, por comportamento conhecido do mPDF, uma tag `

Recomendação: Aplicar `htmlspecialchars(\$\_POST['campo'], ENT\_QUOTES, 'UTF-8')` em todo valor antes de interpolar no HTML; desabilitar busca remota de imagens no mPDF ou restringir a um allowlist; adicionar flags de cookie seguras.

[XSS-4] innerHTML vinculado a strings de tradução estáticas (não explorável hoje, mas padrão frágil)

BAIXO

ACHADO · Arquivo(s): src/app/features/pit-rit/pit-rit.module.ts; src/assets/js/i18n.js — Linha(s): pit-rit.module.ts:23; i18n.js:127

```
<p [innerHTML]="pitRit.description' | translate"></p>
...
if (element.hasAttribute('data-i18n-html')) { element.innerHTML = translation; }
```

Descrição: Ambos os usos vinculam `innerHTML` a valores lidos de arquivos JSON de tradução estáticos (src/assets/i18n/pt-br.json), não a entrada de usuário em tempo de execução. Hoje não há caminho de exploração sem antes comprometer o build/repositório.

Impacto: Baixo hoje; torna-se crítico no dia em que as traduções passarem a vir de um CMS/API editável por usuários.

Recomendação: Preferir interpolação `{{ }}` ou sanitizar explicitamente mesmo para conteúdo 'estático', como reforço de defesa em profundidade.

[XSS-S1] Interpolação padrão do Angular ({{ }}) usada corretamente para campos de texto simples

INFORMATIVO

PONTO FORTE · Arquivo(s):

src/app/features/revista/components/artigo-detail/artigo-detail.component.ts — Linha(s): 39-41

```
<h1>{{ artigo.title }}</h1>
<p class="authors" *ngIf="artigo.authors">{{ artigo.authors }}</p>
<p class="summary" *ngIf="artigo.summary">{{ artigo.summary }}</p>
```

Descrição: Título, autores e resumo usam interpolação padrão do Angular (escapa HTML automaticamente); apenas o campo `content` (rich text) usa o caminho perigoso documentado em XSS-1.

Impacto: O problema está isolado ao conteúdo rico dos editores Quill, não é um padrão generalizado em toda a aplicação.

Recomendação: Ao corrigir XSS-1, usar este arquivo como referência do padrão seguro já predominante.

[XSS-S2] Whitelist estrita de MIME+extensão no upload bloqueia .html/.svg/.js

INFORMATIVO

PONTO FORTE · Arquivo(s): backend/src/routes/upload.routes.ts — Linha(s): 23-36, 60-68

```
const ALLOWED = new Map([
  ['image/jpeg', ['.jpg', '.jpeg']],
  ['image/png', ['.png']],
  ['application/pdf', ['.pdf']],
  ...
]);
// fileFilter rejeita qualquer mimetype/extensão fora da allowlist
```

Descrição: O módulo de upload só aceita um conjunto explícito de tipos de imagem/documento/vídeo, exigindo que MIME type E extensão batam simultaneamente — bloqueando uploads de `.html`/`.svg`/`.js` que poderiam ser servidos como XSS armazenado via `/uploads/`.

Impacto: Fecha um vetor de XSS comum (upload de arquivo ativo) que muitas aplicações similares deixam aberto.

Recomendação: Manter; considerar também validar magic bytes (não só extensão/MIME declarado pelo cliente).

5. Recomendações Priorizadas

Ordenadas por severidade (Crítico → Alto → Médio → Baixo); dentro de cada nível, na ordem em que aparecem os achados correspondentes.

#	Sev.	ID	Recomendação
1	Crítico	ISO-1	Aplicar <code>`authorizeRoles('admin')`</code> (ou papel equivalente) em todas as rotas de escrita administrativa; redesenhar o seed para atribuir 'admin' apenas a contas explícitas; adicionar teste de regressão que falhe se uma rota de escrita não tiver <code>`authorizeRoles`</code> .
2	Crítico	PERM-1	Adicionar <code>`authorizeRoles('admin')`</code> (ou papel adequado) em cada rota de escrita, espelhando exatamente a regra usada em <code>`role.guard.ts`</code> ; cobrir com teste de integração que autentica como usuário comum e espera 403.
3	Crítico	KEY-1	Gerar senha aleatória no primeiro seed e imprimi-la uma única vez no log (ou exigir troca no primeiro login); nunca hardcodar credenciais reais em seeds versionados; adicionar guarda <code>`if (NODE_ENV==='production') throw`</code> para seeds de dados de teste.
4	Crítico	XSS-1	Sanitizar <code>`content`</code> no backend antes de persistir (ex.: <code>`sanitize-html`/sanitizador Quill-aware com allowlist de tags`</code> ); no frontend, trocar <code>`bypassSecurityTrustHtml`</code> por <code>`DomSanitizer.sanitize(SecurityContext.HTML, content)`</code> ou aplicar <code>DOMPurify</code> explicitamente antes do bind; mover o JWT para cookie <code>httpOnly+Secure+SameSite</code> .
5	Alto	ISO-2	Assinar/verificar refresh tokens com <code>`config.jwt.refreshSecret`</code> ; incluir um claim <code>`type`</code> e validá-lo em cada endpoint; considerar rotacionar o refresh token a cada uso.
6	Alto	IDOR-1	Comparar <code>`upload.uploaded_by === req.user.id`</code> (ou exigir papel admin) antes de apagar; validar <code>`id`</code> numérico antes da query.
7	Alto	IDOR-2	Restringir campos sensíveis em <code>`getById`</code> a usuários com papel administrativo/gestão de visitas; ou exigir que o UUID completo seja tratado como capability token de uso único, não reexposto na listagem pública.
8	Alto	KEY-2	Remover o fallback; fazer o compose falhar explicitamente se <code>`DB_PASSWORD`</code> não estiver definido (<code>`\${DB_PASSWORD:?obrigatório}`</code>).
9	Alto	KEY-3	Não copiar <code>`.env.example`</code> para <code>`.env`</code> na imagem final; se um arquivo de exemplo for necessário, nomear diferente de <code>`.env`</code> para que o dotenv nunca o carregue automaticamente; validar no boot que segredos críticos não correspondem aos valores literais do <code>`.env.example`</code> .
10	Médio	ISO-3	Adicionar uma tabela/coluna de vínculo usuário↔laboratório e validar, no controller, que <code>`req.user.id`</code> coordena o <code>`lab_id`</code> informado (ou tem papel admin).
11	Médio	ISO-4	Não publicar essas portas diretamente (usar rede interna do Compose + proxy autenticado, ou túnel/VPN); remover o fallback <code>`:-admin`</code> e exigir <code>`GRAFANA_PASSWORD`</code> obrigatório.
12	Médio	PERM-2	Aplicar <code>`authorizeRoles('admin')`</code> também nessas rotas de leitura administrativa.
13	Médio	IDOR-3	Mesma correção de PERM-1/ISO-3: introduzir papel/vínculo específico para gestão de visitas.
14	Médio	XSS-2	Adicionar os mesmos headers de segurança (CSP, X-Content-Type-Options, etc.) diretamente no bloco <code>`server`</code> do Nginx que serve <code>`/`</code> , ou roteá-lo através do Express.
15	Médio	XSS-3	Aplicar <code>`htmlspecialchars(\$_POST['campo'], ENT_QUOTES, 'UTF-8')`</code> em todo valor antes de interpolar no HTML; desabilitar busca remota de imagens no mPDF ou restringir a um allowlist; adicionar flags de cookie seguras.
16	Baixo	ISO-5	Implementar blacklist/allowlist de refresh tokens no Redis já disponível, invalidada no logout e na troca de senha.
17	Baixo	XSS-4	Preferir interpolação <code>`{{ }}`</code> ou sanitizar explicitamente mesmo para conteúdo 'estático', como reforço de defesa em profundidade.

6. Anexo — Templates de Issues para o GitHub

Um template Markdown pronto para copiar em "New Issue" no GitHub para cada achado do tipo fraqueza com severidade Crítico, Alto ou Médio.

ISO-1 — Nenhum isolamento real de privilégios: todo usuário autenticado é tratado como admin

```
# [SECURITY][CRÍTICO] Nenhum isolamento real de privilégios: todo usuário autenticado é
tratado como admin

**Labels sugeridas:** security, isolamento, severity-critico

## Categoria
Isolamento

## Severidade
Crítico

## Descrição
O seed padrão (executado no setup documentado no README) atribui roles ['user','admin'] a
TODOS os ~160 usuários da lista `allowedUsers` e também ao admin mestre. Não existe nenhum
usuário 'somente leitura' ou 'somente user'. Além disso, o middleware `authorizeRoles()`
(backend/src/middleware/auth.middleware.ts:62) que faria a checagem de papel nunca é
importado ou usado em nenhuma rota (confirmado via busca em todo backend/src). Ou seja: o
conceito de 'papel' existe no schema (coluna jsonb roles) mas não existe em nenhum ponto
de aplicação real.

## Impacto
Qualquer uma das ~160 contas de servidores do IFCE (ou a conta admin mestre) tem, na
prática, controle total sobre boletins, laboratórios, revista e aprovação de visitas
escolares. Não há segmentação de menor privilégio.

## Evidência
- `backend/src/database/seeds/001_initial_data.ts` – linha(s) `12-13, 799-822`
...

const adminPasswordHash = await bcrypt.hash(/* segredo real – redigido, ver linha citada */, 12);
... roles: JSON.stringify(['user', 'admin']), // All allowed to edit bulletins
...

## Recomendação
Aplicar `authorizeRoles('admin')` (ou papel equivalente) em todas as rotas de escrita
administrativa; redesenhar o seed para atribuir 'admin' apenas a contas explícitas;
adicionar teste de regressão que falhe se uma rota de escrita não tiver `authorizeRoles`.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

PERM-1 — Rotas de escrita administrativas exigem apenas login (authMiddleware), nunca papel (authorizeRoles)

```
# [SECURITY][CRÍTICO] Rotas de escrita administrativas exigem apenas login
(authMiddleware), nunca papel (authorizeRoles)

**Labels sugeridas:** security, permissões-frontend-vs-backend, severity-critico

## Categoria
Permissões Frontend vs Backend

## Severidade
Crítico

## Descrição
O frontend Angular usa `RoleGuard`/`AuthService.hasAnyRole()`
(src/app/core/guards/role.guard.ts) para esconder as telas de administração de quem não
tem o papel esperado. Porém, nenhuma das rotas Express equivalentes chama
`authorizeRoles(...)`; todas usam apenas `authMiddleware`, que só verifica 'existe um JWT
válido'. Isso quebra a premissa de que o frontend e o backend aplicam a mesma política.

## Impacto
Qualquer conta autenticada pode chamar diretamente (curl/Postman) `POST/PUT/DELETE
/api/laboratorios`, `/api/boletins` e `/api/revista/...`, contornando totalmente a UI
administrativa, mesmo que o RoleGuard nunca mostre esses botões para ela.

## Evidência
- `backend/src/routes/laboratorios.routes.ts` – linha(s) `9-11`
- `backend/src/routes/boletim.routes.ts` – linha(s) `215,259,297`
- `backend/src/routes/revista.routes.ts` – linha(s) `119,163,209,261,307,334`

...

router.post('/', authMiddleware, laboratorioController.create);
router.put('/:id', authMiddleware, laboratorioController.update);
router.delete('/:id', authMiddleware, laboratorioController.delete);
// idem em boletim.routes.ts e revista.routes.ts – nunca 'authorizeRoles(...)'
...

## Recomendação
Adicionar `authorizeRoles('admin')` (ou papel adequado) em cada rota de escrita,
espelhando exatamente a regra usada em `role.guard.ts`; cobrir com teste de integração que
autentica como usuário comum e espera 403.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

KEY-1 — Senha de admin mestre hardcoded no seed padrão (matrícula 123456)

```
# [SECURITY][CRÍTICO] Senha de admin mestre hardcoded no seed padrão (matrícula 123456)

**Labels sugeridas:** security, chaves-expostas, severity-critico

## Categoria
Chaves Expostas

## Severidade
Crítico

## Descrição
O seed é executado como etapa padrão de instalação (README.md, seção 'Build do Backend', comando `npm run seed`), sem nenhuma checagem de `NODE_ENV`. A senha em texto plano (valor real redigido neste relatório – ver linha citada no repositório) fica permanentemente no histórico do repositório público `c1c3ru/deppi_novo`, associada a uma matrícula fixa e conhecida ('123456').

## Impacto
Qualquer pessoa com acesso de leitura ao repositório (público) conhece uma credencial válida de administrador caso o seed seja aplicado em produção sem troca de senha posterior.

## Evidência
- `backend/src/database/seeds/001_initial_data.ts` – linha(s) `13, 811-822`

...
const adminPasswordHash = await bcrypt.hash(/* segredo real – redigido, ver linha citada */, 12);
...
usersToInsert.push({
  registration: '123456',
  name: 'Administrador Sistema',
  password_hash: adminPasswordHash,
  roles: JSON.stringify(['user', 'admin']),
  ...
});
`);
`);

## Recomendação
Gerar senha aleatória no primeiro seed e imprimi-la uma única vez no log (ou exigir troca no primeiro login); nunca hardcodar credenciais reais em seeds versionados; adicionar guarda `if (NODE_ENV==='production') throw` para seeds de dados de teste.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

XSS-1 — Stored XSS via conteúdo do editor Quill em Boletins e Revista (bypassSecurityTrustHtml sem nenhuma sanitização)

```
# [SECURITY][CRÍTICO] Stored XSS via conteúdo do editor Quill em Boletins e Revista
(bypassSecurityTrustHtml sem nenhuma sanitização)

**Labels sugeridas:** security, xss, severity-critico

## Categoria
XSS

## Severidade
Crítico

## Descrição
`bypassSecurityTrustHtml` faz exatamente o oposto do que o comentário do código diz:
desativa o sanitizador automático do Angular para aquele valor. O backend também nunca
sanitiza `content` ao salvar (nem ao criar, nem ao atualizar). Uma busca por
`DOMPurify`/`dompurify` em `src/app` não encontra nenhum uso – a dependência só existe
como `overrides` transitivo em package.json, nunca chamada pela aplicação. O mesmo padrão
se repete em `artigo-detail.component.ts` para artigos da Revista.

## Impacto
Combinado com PERM-1 (qualquer uma das ~160 contas autenticadas pode publicar
boletins/artigos) e com o armazenamento do JWT em localStorage
(src/app/core/services/storage.service.ts), um payload como `
```

ISO-2 — Access token e refresh token compartilham o mesmo segredo (JWT_REFRESH_SECRET nunca é usado)

```
# [SECURITY][ALTO] Access token e refresh token compartilham o mesmo segredo
(JWT_REFRESH_SECRET nunca é usado)

**Labels sugeridas:** security, isolamento, severity-alto

## Categoria
Isolamento

## Severidade
Alto

## Descrição
`backend/src/config/environment.ts:42-47` define `config.jwt.refreshSecret` separadamente,
mas em nenhum lugar do código esse valor é lido — tanto o access token quanto o refresh
token de 30 dias são assinados e verificados com `config.jwt.secret`. Nenhum claim (`type:
'access' | 'refresh'`) distingue os dois. Isso significa que um access token de curta
duração pode ser reenviado como refresh token em POST /auth/refresh, e que comprometer o
segredo único compromete ambos os tipos de token permanentemente, sem possibilidade de
rotacionar um sem o outro.

## Impacto
Perde-se o isolamento de blast-radius entre tokens de curta e longa duração; incidentes de
resposta (revogar só refresh tokens) tornam-se impossíveis.

## Evidência
- `backend/src/routes/auth.routes.ts` — linha(s) `115-123, 179-182`
...
const accessToken = jwt.sign(..., config.jwt.secret, { expiresIn: '1h' });
const refreshToken = jwt.sign({ id: user.id }, config.jwt.secret, { expiresIn: '30d' });
// /auth/refresh:
const decoded = jwt.verify(refreshToken, config.jwt.secret) as jwt.JwtPayload;
...

## Recomendação
Assinar/verificar refresh tokens com `config.jwt.refreshSecret`; incluir um claim `type` e
validá-lo em cada endpoint; considerar rotacionar o refresh token a cada uso.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```


IDOR-1 — DELETE /api/upload/:id apaga qualquer anexo de qualquer usuário — ID sequencial, sem checar proprietário

```
# [SECURITY][ALTO] DELETE /api/upload/:id apaga qualquer anexo de qualquer usuário – ID sequencial, sem checar proprietário

**Labels sugeridas:** security, idor, severity-alto

## Categoria
IDOR

## Severidade
Alto

## Descrição
A tabela `uploads` usa `table.increments('id')` (backend/src/database/migrations/005_create_uploads_table.ts:5) – IDs inteiros sequenciais e previsíveis (1, 2, 3, ...). A coluna `uploaded_by` existe no schema, mas o handler de DELETE nunca a compara com `req.user.id`; só exige que exista um JWT válido (o router inteiro fica atrás de `authMiddleware` aplicado em backend/src/index.ts:127).

## Impacto
Qualquer conta autenticada pode iterar `DELETE /api/upload/1`, `/2`, `/3`... e desativar anexos de boletins/laboratórios que não enviou, quebrando imagens/documentos publicados por outras pessoas.

## Evidência
- `backend/src/routes/upload.routes.ts` – linha(s) `170-179`

```
router.delete('/:id', async (req: Request, res: Response) => {
 const { id } = req.params;
 await db('uploads').where({ id }).update({ is_active: false });
 res.json({ success: true, message: 'Anexo removido' });
});
```

## Recomendação
Comparar `upload.uploaded_by === req.user.id` (ou exigir papel admin) antes de apagar; validar `id` numérico antes da query.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

IDOR-2 — GET /api/visitas/:id expõe PII completa de escolas sem checar relação do requisitante com a visita

```
# [SECURITY][ALTO] GET /api/visitas/:id expõe PII completa de escolas sem checar relação
do requisitante com a visita

**Labels sugeridas:** security, idor, severity-alto

## Categoria
IDOR

## Severidade
Alto

## Descrição
`schooI_visits.id` é UUID (mitiga adivinhação à força bruta – ponto positivo), mas o
próprio endpoint público não autenticado `GET /api/visitas` já devolve esse UUID em
`toPublicVisit()` para visitas confirmadas. A partir daí, `GET /api/visitas/:id` (só
`authMiddleware`, sem checar papel ou vínculo com a visita) devolve o registro completo:
`contact_email`, `contact_phone`, `director_name`, `whatsapp_phone`,
`institutional_email`, `notes`, etc.

## Impacto
Qualquer uma das ~160 contas autenticadas pode pegar um UUID visto anonimamente na agenda
pública e obter dados de contato pessoal de diretores/responsáveis escolares, sem qualquer
relação legítima com aquele agendamento específico.

## Evidência
- `backend/src/controllers/visit.controller.ts` – linha(s) `12-22, 77-103`

...

function toPublicVisit(visit) {
  return { id: visit.id, school_name: ..., ... }; // inclui 'id'
}
...
async getById(req, res, next) {
  const { id } = req.params;
  const visit = await db('school_visits').where({ id }).first();
  ...
  res.json({ ...visit, labs });
}
...

## Recomendação
Restringir campos sensíveis em `getById` a usuários com papel administrativo/gestão de
visitas; ou exigir que o UUID completo seja tratado como capability token de uso único,
não reexposto na listagem pública.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

KEY-2 — Senha do PostgreSQL com fallback hardcoded no docker-compose.yml versionado

```
# [SECURITY][ALTO] Senha do PostgreSQL com fallback hardcoded no docker-compose.yml
versionado

**Labels sugeridas:** security, chaves-expostas, severity-alto

## Categoria
Chaves Expostas

## Severidade
Alto

## Descrição
Caso a variável de ambiente `DB_PASSWORD` não esteja definida no momento de `docker-
compose up`, o Postgres é iniciado com uma senha literal fixa (valor real redigido neste
relatório – ver linha citada no repositório), publicada no repositório.

## Impacto
Uma instalação apressada (sem `.env` completo) sobe um banco com senha conhecida
publicamente.

## Evidência
- `docker-compose.yml` – linha(s) `26, 68`

...
- POSTGRES_PASSWORD=${DB_PASSWORD:-«segredo redigido»} # Fallback caso a variável falhe
...

## Recomendação
Remover o fallback; fazer o compose falhar explicitamente se `DB_PASSWORD` não estiver
definido (`${DB_PASSWORD:?obrigatório}`).

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

KEY-3 — Dockerfile grava .env.example (com segredos-placeholder públicos) como .env real dentro da imagem publicada

```
# [SECURITY][ALTO] Dockerfile grava .env.example (com segredos-placeholder públicos) como
.env real dentro da imagem publicada

**Labels sugeridas:** security, chaves-expostas, severity-alto

## Categoria
Chaves Expostas

## Severidade
Alto

## Descrição
A imagem `ghcr.io/deppimaracanau/deppi_novo:main` (referenciada diretamente em docker-
compose.yml:6) contém, dentro dela, um arquivo `/app/.env` que é cópia literal do
`.env.example` público. `dotenv.config()` só preenche uma chave que AINDA NÃO existe em
`process.env`. Se o container for executado sem TODAS as variáveis passadas explicitamente
(ex.: `docker run` direto da imagem publicada, sem o docker-compose.yml completo),
`process.env.JWT_SECRET` é preenchido pelo dotenv com o placeholder público
`your_super_secret_jwt_key_here_change_in_production`. Como esse valor é uma string não-
vazia, o guard `requireEnv` (pensado para falhar em produção sem segredo) é satisfeito
silenciosamente por um segredo já conhecido publicamente.

## Impacto
Em um deploy que não injete `JWT_SECRET` de forma explícita, a aplicação assina JWTs de
produção com um segredo publicado no GitHub – permitindo forjar tokens de qualquer
usuário/admin.

## Evidência
- `Dockerfile` – linha(s) `58`
- `backend/src/config/environment.ts` – linha(s) `9-15`
- `.env.example` – linha(s) `21,23`
...

# Dockerfile
COPY --from=builder --chown=nextjs:nodejs /app/.env.example /app/.env

# .env.example
JWT_SECRET=your_super_secret_jwt_key_here_change_in_production

# environment.ts
function requireEnv(key, fallback) {
  const value = process.env[key] || fallback;
  if (!value && process.env.NODE_ENV === 'production') throw new Error(...);
  return value ?? '';
}
...

## Recomendação
Não copiar `.env.example` para `.env` na imagem final; se um arquivo de exemplo for
necessário, nomear diferente de `.env` para que o dotenv nunca o carregue automaticamente;
validar no boot que segredos críticos não correspondem aos valores literais do
`.env.example`.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

ISO-3 — Laboratórios não têm proprietário/coordenador — qualquer usuário controla a disponibilidade de qualquer lab

```
# [SECURITY][MÉDIO] Laboratórios não têm proprietário/coordenador – qualquer usuário controla a disponibilidade de qualquer lab

**Labels sugeridas:** security, isolamento, severity-médio

## Categoria
Isolamento

## Severidade
Médio

## Descrição
A tabela `laboratorios` não possui nenhuma coluna que associe um laboratório a um coordenador/responsável. O endpoint `PATCH /api/visitas/:id/labs/:lab_id` (protegido só por `authMiddleware`) permite que QUALQUER usuário autenticado marque QUALQUER laboratório como disponível/indisponível para QUALQUER visita, sem vincação ao seu próprio laboratório.

## Impacto
Um funcionário de um laboratório pode confirmar ou recusar disponibilidade em nome de outro laboratório, inclusive influenciando o e-mail de confirmação automático enviado à escola.

## Evidência
- `backend/src/database/migrations/006_create_laboratorios_table.ts` – linha(s) `4-13`
- `backend/src/controllers/visit.controller.ts` – linha(s) `272-292`

...

// migration: laboratorios não tem coluna owner/coordinator_id
// controller:
async updateLabAvailability(req, res, next) {
  const { id, lab_id } = req.params;
  const { status } = req.body;
  const updated_by_user_id = (req as any).user?.id || null;
  const [updated] = await db('visit_lab_availability')
    .where({ visit_id: id, lab_id })
    .update({ status, updated_by_user_id, ... });
  ...
}

## Recomendação
Adicionar uma tabela/coluna de vínculo usuário↔laboratório e validar, no controller, que `req.user.id` coordena o `lab_id` informado (ou tem papel admin).

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

ISO-4 — Prometheus exposto sem autenticação e Grafana com senha padrão publicados diretamente no host

```
# [SECURITY][MÉDIO] Prometheus exposto sem autenticação e Grafana com senha padrão
publicados diretamente no host

**Labels sugeridas:** security, isolamento, severity-médio

## Categoria
Isolamento

## Severidade
Médio

## Descrição
Prometheus (sem autenticação nativa) e Grafana são publicados diretamente em portas do
host, fora do Nginx/reverse proxy. Se `GRAFANA_PASSWORD` não for definido no ambiente, o
Grafana sobe com a senha padrão 'admin', divulgada no próprio docker-compose.yml
versionado.

## Impacto
Exposição de métricas internas (Prometheus) e possível acesso administrativo ao Grafana
(dashboards, datasources) para qualquer host com rota de rede até as portas 9090/3001.

## Evidência
- `docker-compose.yml` – linha(s) `119-120, 139-143`

...
prometheus:
  ports: ["9090:9090"]
...
grafana:
  ports: ["3001:3000"]
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD:-admin}
...

## Recomendação
Não publicar essas portas diretamente (usar rede interna do Compose + proxy autenticado,
ou túnel/VPN); remover o fallback `:-admin` e exigir `GRAFANA_PASSWORD` obrigatório.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

PERM-2 — Listagem administrativa de boletins/edições (rascunhos inclusos) exige só autenticação, não papel

[SECURITY][MÉDIO] Listagem administrativa de boletins/edições (rascunhos inclusos) exige só autenticação, não papel

Labels sugeridas: security, permissões-frontend-vs-backend, severity-médio

Categoria

Permissões Frontend vs Backend

Severidade

Médio

Descrição

Os endpoints '/admin/all' retornam boletins e edições em qualquer status (incluindo 'draft'), protegidos só por 'authMiddleware'. Qualquer conta autenticada vê conteúdo ainda não publicado.

Impacto

Vazamento de informação (rascunhos internos) para contas que a UI nunca deveria expor a essas telas.

Evidência

- 'backend/src/routes/boletim.routes.ts' – linha(s) '16-18'

- 'backend/src/routes/revista.routes.ts' – linha(s) '45-48'

...

```
router.get('/admin/all', authMiddleware, async (req, res) => { ... });
```

```
router.get('/edicoes/admin/all', authMiddleware, async (req, res) => { ... });
```

...

Recomendação

Aplicar 'authorizeRoles('admin')' também nessas rotas de leitura administrativa.

Checklist de aceite

- [] Correção implementada conforme a recomendação acima

- [] Teste automatizado cobrindo o caso (regressão)

- [] Validado manualmente em ambiente de homologação

- [] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório

- [] Revisão de código aprovada

IDOR-3 — PATCH /api/visitas/:id/status e /:id/labs/:lab_id alteram qualquer visita/lab sem checar posse

```
# [SECURITY][MÉDIO] PATCH /api/visitas/:id/status e /:id/labs/:lab_id alteram qualquer
visita/lab sem checar posse

**Labels sugeridas:** security, idor, severity-médio

## Categoria
IDOR

## Severidade
Médio

## Descrição
Assim como ISO-3, estes endpoints só exigem `authMiddleware`. Qualquer conta pode
confirmar/cancelar qualquer visita escolar (o que dispara e-mail automático ao contato da
escola e cria evento no Google Calendar institucional).

## Impacto
Ações de negócio sensíveis (aprovar visita, notificar escola, ocupar agenda) não estão
restritas a quem deveria geri-las.

## Evidência
- `backend/src/controllers/visit.controller.ts` — linha(s) `195-270, 272-292`
...
async updateStatus(req, res, next) {
  const { id } = req.params;
  const { status } = req.body;
  const visit = await db('school_visits').where({ id }).first();
  ... // nenhuma checagem de que req.user está autorizado para ESTA visita
...

## Recomendação
Mesma correção de PERM-1/ISO-3: introduzir papel/vínculo específico para gestão de
visitas.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```


XSS-2 — CSP do Helmet nunca alcança a página real (SPA é servida direto pelo Nginx, sem os mesmos headers)

```
# [SECURITY][MÉDIO] CSP do Helmet nunca alcança a página real (SPA é servida direto pelo
Nginx, sem os mesmos headers)

**Labels sugeridas:** security, xss, severity-médio

## Categoria
XSS

## Severidade
Médio

## Descrição
Em produção, o Nginx serve `index.html` e os bundles JS/CSS diretamente do disco (`root
/app/dist/deppi`), sem passar pelo Express/Helmet. O CSP restritivo configurado
(`scriptSrc: ['"self""]`) só é enviado em respostas JSON de `/api/*`, que o navegador
nunca renderiza como HTML – ou seja, a página onde um XSS (XSS-1) realmente executaria não
recebe CSP nenhum.

## Impacto
Uma camada de defesa em profundidade que mitigaria/bloquearia a execução do payload de
XSS-1 está configurada, mas nunca chega a ser aplicada onde importa.

## Evidência
- `backend/src/index.ts` – linha(s) `60-77`
- `nginx.conf` – linha(s) `30-39`

...

// index.ts – só decora respostas do Express (/api/*, /health)
this.app.use(helmet({
  contentSecurityPolicy: { directives: { scriptSrc: ['"self"'], ... } }
}));

// nginx.conf – serve o SPA direto do disco, sem headers equivalentes
location / {
  try_files $uri $uri/ /index.html; // root /app/dist/deppi
}
...

## Recomendação
Adicionar os mesmos headers de segurança (CSP, X-Content-Type-Options, etc.) diretamente
no bloco `server` do Nginx que serve `/`, ou roteá-lo através do Express.

## Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado cobrindo o caso (regressão)
- [ ] Validado manualmente em ambiente de homologação
- [ ] Sem novas ocorrências do mesmo padrão em outros arquivos do repositório
- [ ] Revisão de código aprovada
```

XSS-3 — Módulo legado PHP (cargahoraria) interpola \$_POST direto em HTML enviado ao mPDF, sem escaping

```
# [SECURITY][MÉDIO] Módulo legado PHP (cargahoraria) interpola $_POST direto em HTML
enviado ao mPDF, sem escaping

**Labels sugeridas:** security, xss, severity-médio

## Categoria
XSS

## Severidade
Médio

## Descrição
Nenhuma ocorrência de `htmlspecialchars`/`htmlentities` existe no arquivo (confirmado por
busca). Todos os campos do formulário PIT (nome, SIAPE, e-mail, curso, campus, etc.) vão
direto de `$_POST` para uma string HTML montada por concatenação e passada a
`mPDF::WriteHTML()`. Os mesmos valores também são gravados via `setcookie()` sem as flags
`HttpOnly`/`Secure`/`SameSite`.

## Impacto
Um valor malicioso injeta HTML arbitrário no PDF gerado (falsificação de conteúdo de um
documento institucional) e, por comportamento conhecido do mPDF, uma tag `
```